

TUGAS
SISTEM OPERASI



Disusun oleh :
Muhammad Rafi Hendardi
2410651062

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

BAB I

PEMBAHASAN

Tugas 1 :

- Mengapa program mengalami deadlock?
Karena Thread 1 mengunci lock1 dan menunggu lock2 dan Thread 2 mengunci lock 2 dan menunggu lock1, sehingga keduanya saling menunggu secara permanen.
- Kondisi deadlock apa saja yang terpenuhi?
 1. Mutual Exclusion : karena mutex hanya bisa dipegang oleh satu thread saja.
 2. Hold and Wait : karena thread memegang satu lock sambil menunggu dari lock lainnya.
 3. No preemption : karena lock tidak bisa dipaksa dilepas
 4. Circular Wait : karena Thread 1 membutuhkan lock2, dan thread 2 membutuhkan lock1
- Gambar diagram resource allocation graph-nya
T1 -> lock1 -> T2 -> lock2 -> Kembali ke T1

Tugas 2 :

- Lock Ordering

```
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_ordering$ nano deadlock_ordering.c
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_ordering$ gcc deadlock_ordering.c -o deadlock_ordering
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_ordering$ ./deadlock_ordering
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===

Thread 1: Mengambil Lock 1...
Thread 2: Mengambil Lock 1...
Thread 1: Mengambil Lock 2...
Thread 1: Selesai bekerja
Thread 2: Mengambil Lock 2...
Thread 2: Selesai bekerja

=== PROGRAM SELESAI TANPA DEADLOCK ===
```

1. Semua thread wajib mengambil lock dengan urutan dari yang terkecil ke terbesar.
 2. Circular wait tidak mungkin terjadi
- Try-lock dengan retry

```
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_lock$ nano deadlock_lock.c
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_lock$ gcc deadlock_lock.c -o deadlock_lock
rafi@DESKTOP-TM81UKJ:~/Praktikum6/deadlock_lock$ ./deadlock_lock
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===

Thread 1: Mengambil Lock 1...
Thread 2: Mengambil Lock 1...
Thread 1: Mengambil Lock 2...
Thread 1: Selesai bekerja
Thread 2: Mengambil Lock 2...
Thread 2: Selesai bekerja

=== PROGRAM SELESAI TANPA DEADLOCK ===
```

1. Jika resource tidak tersedia, thread melepas lock dan mencoba lagi.
 2. Tidak pernah menunggu selamanya.
- Bandingkan kedua metode tersebut

1. Lock Ordering : Paling efisien, tidak retry dan harus dipatuhi semua thread.
2. Try-Lock : Tidak butuh aturan urutan dan bisa spin lama.

Tugas 3 :

```
rafi@DESKTOP-TM81UKJ:~/Praktikum6/bankers_custom$ nano bankers_custom.c
rafi@DESKTOP-TM81UKJ:~/Praktikum6/bankers_custom$ gcc bankers_custom.c -o bankers_custom
rafi@DESKTOP-TM81UKJ:~/Praktikum6/bankers_custom$ ./bankers_custom
```

```
=== STATUS SISTEM ===

Available: 5 4 3

Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1

=== MEMERIKSA SAFETY ===

[SUKSES] Sistem aman. Safe Sequence: P1 → P2 → P3 → P0

=== REQUEST DARI P1 ===
Meminta: 1 0 2

=== MEMERIKSA SAFETY ===
```

```
rafi@DESKTOP-TM81UKJ: ~/Praktikum6/bankers_custom

[SUKSES] Sistem aman. Safe Sequence: P1 → P2 → P3 → P0
[DITERIMA] Request aman dan dialokasikan

=== STATUS SISTEM ===

Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1

=== PROGRAM SELESAI ===
```

Tugas 4 :

```
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ nano tugas4.c
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ gcc tugas4.c -o tugas4
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ ./tugas4
```

```
=== SIMULASI DEADLOCK TRANSFER ===

B: Mengunci Rekening 2
A: Mengunci Rekening 1
B: Mencoba mengunci Rekening 1
A: Mencoba mengunci Rekening 2
^C
```

Solusi dari deadlock diatas :

```
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ nano tugas4.c
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ gcc tugas4.c -o tugas4
rafi@DESKTOP-TM81UKJ:~/Praktikum6/tugas4$ ./tugas4

=== SOLUSI DEADLOCK TRANSFER (LOCK ORDERING) ===

B: Mengunci rekening secara terurut
B: Transfer berhasil
A: Mengunci rekening secara terurut
A: Transfer berhasil

=== PROGRAM SELESAI TANPA DEADLOCK ===
```